FLE TRIBE

WEEK 13 · TUESDAY · LESSON GUIDE

Forms

This week: Decide where state lives, build forms, and share data across your app.

Controlled inputs

In React, form fields are usually **controlled**: the value lives in state and every keystroke updates it.

```
const [email, setEmail] = useState('');

<input type="email" value={email} onChange={(e) => setEmail(e.target.value)} />
```

Validation that feels friendly

- Validate on submit, then live as the user fixes each field.
- Show errors only after a field has been touched, not while someone is still typing their first letters.
- Put the message right under the field and say how to fix it: "Enter a date that isn't in the past".
- Disable the submit button while saving and show progress ("Saving…").

Example: due date can't be in the past

```
function validateDueDate(value) {
  if (!value) return 'Choose a due date.';
  const today = new Date().toISOString().slice(0, 10);
  return value < today ? 'Choose a date that isn\'t in the past.' : '';
}
```

When to use a form library

For one or two fields, plain state is fine. For long forms with many rules, **React Hook Form** handles values, touched state and validation for you, with less re-rendering.

Research questions

Answer these in the app, in your own words, to unlock the next lesson.

1. When would you reach for a form library?
2. Write a validator for "due date can't be in the past".

Start your research here

- react.dev: https://react.dev/reference/react-dom/components/input
- React Hook Form: https://react-hook-form.com/get-started